Labo1: Getting Acquainted with GPU Programming

Arthur Valentin, Joeri Sevens

October 31, 2024

Overview

Voor het eerste labo was het de bedoeling een array van bits van onbepaalde lengte om te draaien met behulp van de GPU. Op deze manier konden we wennen aan de basisprincipes van GPU-programmering.

Methodology

Ons idee is om elke thread de array vanaf de achterkant te laten lezen en de waarden vanaf het begin weg te schrijven, zodat de array wordt omgedraaid. Om elke thread een unieke positie te geven, berekenen we de index door het block ID te vermenigvuldigen met de blockgrootte en daar het thread ID bij op te tellen. Dit wordt geïllustreerd in de vecFlip functie in onze code.

Results

Onze methodologie was correct, maar er ontbraken enkele belangrijke stappen die typisch zijn voor GPU-programmering. Deze omvatten het toewijzen van geheugen, het kopiëren van de array naar het gealloceerde geheugen, het wachten op de GPU om de berekeningen te voltooien, het terugschrijven van de resultaten naar de host en het vrijgeven van het toegewezen geheugen.

Conclusion

We hebben verschillende basisprincipes van GPU-programmering onder de knie gekregen.

```cpp
#include <stdio.h>
#include <cuda_runtime.h>

__global__ void vecFlip(int* A, int* B, int N) {
    int i =  blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N) {
        B[i] = A[N-i-1];
    }
}

int main() {
    int N = 64;
    int A[N];
    int B[N];

    for (int i = 0; i < N; i++) {
        A[i] = rand() % 100;
    }

    for (int i = 0; i < N; i++) {
        printf("%d ", A[i]);
    }

    printf("\n");

    int *d_A, *d_B;
    cudaMalloc(&d_A, N * sizeof(int));
    cudaMalloc(&d_B, N * sizeof(int));

    cudaMemcpy(d_A, A, N * sizeof(int), cudaMemcpyHostToDevice);

    int threadsPerBlock = 4;
    int blocksPerGrid = (N + threadsPerBlock - 1) / threadsPerBlock;

    vecFlip<<<blocksPerGrid, threadsPerBlock>>>(d_A, d_B, N);

    cudaDeviceSynchronize();

    cudaMemcpy(B, d_B, N * sizeof(int), cudaMemcpyDeviceToHost);

    for (int i = 0; i < N; i++) {
        printf("%d ", B[i]);
    }

    cudaFree(d_A);
    cudaFree(d_B);

    return 0;
}
```